RUQuant: Towards Refining Uniform Quantization for Large Language Models

Han Liu
Dalian University of Technology
Dalian, China
liu.han.dut@gmail.com

Haotian Gao
Dalian University of Technology
Dalian, China
haotian.dlut@gmail.com

Changya Li
Dalian University of Technology
Dalian, China
lichangya.dut@gmail.com

Feng Zhang
Peking University
Beijing, China
fengzhangyvonne@gmail.com

Xiaotong Zhang*
Dalian University of Technology
Dalian, China
zxt.dut@hotmail.com

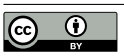
Wei Wang
Macao Polytechnic University
Macao, China
weiwang@mpu.edu.mo

Hong Yu
Dalian University of Technology
Dalian, China
hongyu@dlut.edu.cn

Abstract

The increasing size and complexity of large language models (LLMs) have raised significant challenges in deployment efficiency, particularly under resource constraints. Post-training quantization (PTQ) has emerged as a practical solution by compressing models without requiring retraining. While existing methods focus on uniform quantization schemes for both weights and activations, they often suffer from substantial accuracy degradation due to the non-uniform nature of activation distributions. In this work, we revisit the activation quantization problem from a theoretical perspective grounded in the Lloyd-Max optimality conditions. We identify the core issue as the non-uniform distribution of activations within the quantization interval, which causes the optimal quantization point under the Lloyd-Max criterion to shift away from the midpoint of the interval. To address this issue, we propose a two-stage orthogonal transformation method, RUQuant. In the first stage, activations are divided into blocks. Each block is mapped to uniformly sampled target vectors using composite orthogonal matrices, which are constructed from Householder reflections and Givens rotations. In the second stage, a global Householder reflection is fine-tuned to further minimize quantization error using Transformer output discrepancies. Empirical results show that our method achieves near-optimal quantization performance without requiring model fine-tuning: RUQuant achieves 99.8% of full-precision accuracy with W6A6 and 97% with W4A4 quantization for a 13B LLM, within approximately one minute. A fine-tuned variant yields even higher accuracy, demonstrating the effectiveness and scalability of our approach.

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
KDD '26, Jeju Island, Republic of Korea
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2258-5/2026/08
https://doi.org/10.1145/3770854.3780259

CCS Concepts

• **Computing methodologies** → **Natural language processing**.

Keywords

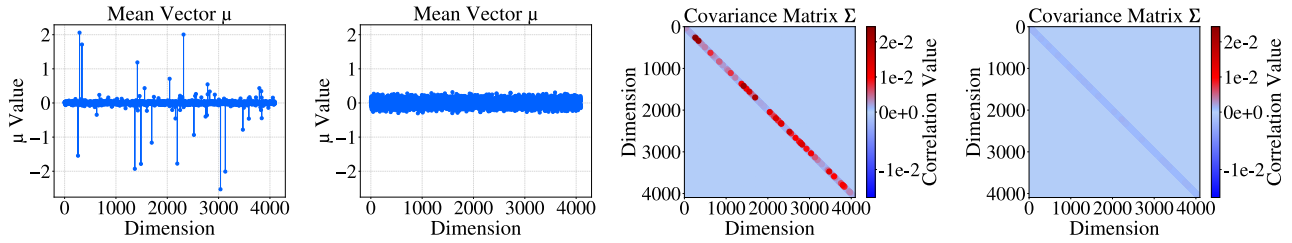
Large Language Models, Post-Training Quantization

ACM Reference Format:
Han Liu, Haotian Gao, Changya Li, Feng Zhang, Xiaotong Zhang, Wei Wang, and Hong Yu. 2026. RUQuant: Towards Refining Uniform Quantization for Large Language Models. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (KDD '26), August 09–13, 2026, Jeju Island, Republic of Korea.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3770854.3780259

1 Introduction

Large language models (LLMs) [1–4] based on the Transformer architecture [5], achieve remarkable success across a wide range of tasks. With the scaling of these models, the computational and memory demands make deployment increasingly challenging, especially under resource-constrained conditions. These challenges underscore the importance of model compression techniques. Among them, post-training quantization (PTQ) is particularly effective, as it compresses models without requiring costly retraining.

Recent PTQ works, such as GPTQ [6], SpQR [7], QuIP [8], and AWQ [9], focus on weight-only quantization for LLMs, converting full-precision weights into low-bit formats to minimize memory usage while achieving nearly lossless 4-bit quantization. To further improve inference efficiency, several works [10–12] shift toward weight-activation quantization, which quantizes both weights and activations into low-bit representations for accelerating matrix multiplications. To ensure hardware efficiency and simplify implementation, most of these methods adopt uniform quantizers. However, empirical evidence shows that activations often exhibit highly non-uniform distributions [12], which can result in significant quantization errors under uniform quantization schemes. While some approaches attempt to address this issue by suppressing outliers and reducing the dynamic range of activations—using techniques such



(a) Original (b) RUQuant processed (c) Original (d) RUQuant processed

Figure 1: The mean and covariance visualizations of activations before and after RUQuant processing. (a) Original mean vector. (b) Mean vector after RUQuant processing. (c) Original covariance matrix. (d) Covariance matrix after RUQuant processing.

as clipping [13], outlier channel scaling [10], or parameter search [14, 15]—these methods are primarily heuristic and cannot systematically resolve the fundamental mismatch between non-uniform activation distributions and uniform quantizers.

In this work, we revisit the problem from a theoretical perspective, based on the Lloyd-Max quantization conditions, which characterize the optimality criteria for quantizer design—the decision boundary condition and the centroid condition, i.e., Eq. (3) and Eq. (4). Our analysis reveals that the performance degradation of uniform quantizers arises primarily from the mismatch between non-uniform activation distributions and the uniform quantization grid. Specifically, in non-uniform distributions, the optimal quantization points (centroids) deviate from the midpoints of quantization intervals, thereby violating the assumptions underlying uniform quantization and resulting in increased quantization error. To overcome this problem, we propose a simple yet effective quantization method, named RUQuant, which facilitates the uniform quantizer by transforming non-uniform activation distributions into approximately uniform ones. This transformation enables the uniform quantizer to achieve performance closer to the theoretical optimum defined by the Lloyd-Max conditions.

Specifically, our proposed RUQuant is a two-stage method that refines uniform quantization through orthogonal transformations, to improve the efficiency and accuracy when deploying LLMs. In the first stage, to reduce parameter overhead, we partition activations into small blocks, and sample a target vector using a uniform distribution generator for each block. We then construct a composite orthogonal matrix—combining Householder reflections and Givens rotations—by accurately computing a closed-form transformation, such that the original activation vectors are mapped precisely to their corresponding target directions. In the second stage, to achieve uniformity between blocks, we introduce a global refinement step. A single Householder reflection vector is initialized via uniform sampling, and then fine-tuned by optimizing the discrepancy between the outputs of Transformer blocks before and after quantization to minimize quantization error. This step further refines the reflection hyperplane, allowing it to better align with the structure of the activation space.

The main contributions of this work can be summarized as follows: 1) We propose RUQuant, a novel two-stage orthogonal transformation method that effectively transforms non-uniform activation distributions into approximately uniform ones, shown in Figure 1, enabling more accurate uniform quantization without requiring model retraining. 2) Theoretically, we revisit activation quantization through the lens of Lloyd-Max optimality conditions, revealing the core cause of quantization errors in uniform schemes. 3) Experimentally, RUQuant demonstrates superior performance on LLMs, specifically on a 13B-scale model, achieving near full-precision accuracy in one minute without fine-tuning[1].

2 Related Work

Quantization reduces model storage and computation by converting floating-point values into lower-bit representations [16, 17]. Post-training quantization (PTQ) is a practical and widely adopted approach that applies quantization after training, without modifying the training pipeline or requiring model retraining [18–23]. Its simplicity and low computational cost make PTQ especially appealing for large language models (LLMs).

To further decrease memory footprint and computational cost, recent efforts extend PTQ to joint weight–activation quantization. However, activation quantization remains challenging due to highly dynamic input distributions and sensitivity to outliers [24]. To address this, a series of methods introduce equivalent transformation modules to reshape weight and activation distributions while preserving model outputs. SmoothQuant [10] first proposes a channel-wise scaling strategy that shifts activation outliers into weights, reducing activation quantization error. Building on this idea, OmniQuant [13] incorporates learnable scaling and bias terms to better smooth weight distributions. AffineQuant [11] further generalizes this approach by employing a learnable affine transformation matrix to minimize quantization error in a more expressive manner. DuQuant [12] introduces dual transformations to suppress both large-magnitude activations and activation outliers, achieving improved robustness under activation quantization. QuaRot [25] leverages randomized Hadamard rotations to remove activation outliers and enable end-to-end 4-bit quantization, while SpinQuant [15] mitigates randomness-induced instability via learnable Cayley-SGD rotations. Most recently, DFRot [26] identifies tokens with massive activations and optimizes rotations specifically for them, achieving improved accuracy in 4-bit settings. Similar equivalent transformation strategies have also been explored beyond quantization, such as in structured pruning and low-rank decomposition [27, 28]. These

[1]The source code is available at: https://github.com/RUQuant-code/RUQuant.

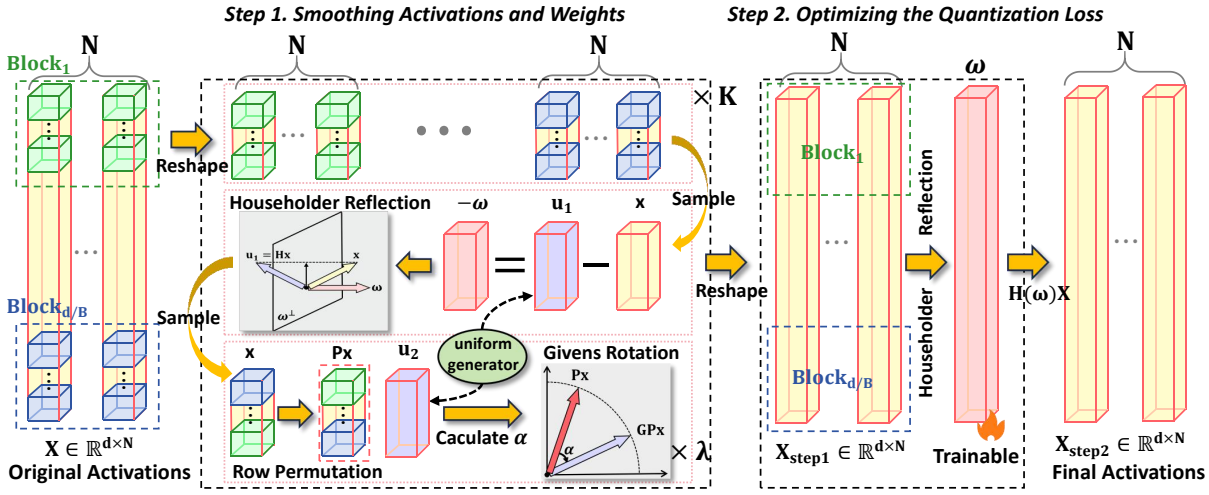


Figure 2: The overall framework of RUQuant (we omitted the zigzag permutation process in the figure). Original activations with size $d \times N$ are reshaped into $B \times \frac{dN}{B}$, all column vectors sharing a common rotation matrix. In Step 1, sampled vector $\mathbf{x} \in \mathbb{R}^B$ is transformed using Householder and Givens rotations based on uniformly generated vectors $\mathbf{u}_1$ and $\mathbf{u}_2$, then reconstructed. In Step 2, a trainable Householder matrix is optimized to minimize quantization loss.

extensions further highlight the generality of transformation-based approaches across model compression paradigms.

3 Motivation

A b-bit uniform quantizer divides the real axis into $N = 2^b$ equal intervals and uses the midpoints of these intervals as quantization points. For an input vector $\mathbf{x}$, the quantized low-bit integer is:

$$\mathbf{x}_q = \left(\left\lfloor \frac{\mathbf{x}}{s} \right\rceil + z \right), \quad s = \frac{\max(\mathbf{x}) - \min(\mathbf{x})}{2^b - 1}, \quad z = -\left\lfloor \frac{\min(\mathbf{x})}{s} \right\rceil, \quad (1)$$

where s is the step size and z is the zero-point. This fixed-step scheme enables efficient integer operations and thus fast inference. While fixed-step uniform quantization offers advantages in computational efficiency and implementation simplicity, its performance heavily depends on a key assumption that the input data is uniformly distributed within the quantization range.

However, when quantizing weights and activations simultaneously, this assumption often fails to hold. In LLMs, activations exhibit highly non-uniform distributions across dimensions, which leads to suboptimal quantization outcomes. To illustrate this, we visualize the statistical properties of activations. As shown in Figure 1(a), the activation means vary significantly across dimensions, ranging approximately from $[-2, 2]$, indicating an uneven distribution of values. Figure 1(c) shows that the covariance matrix is nearly diagonal: most dimensions exhibit low variance, while a few show large fluctuations. This suggests that the activations are weakly correlated but have highly imbalanced scales across channels.

Empirical observation. For a uniform quantizer, such outlier values in the activation distribution can significantly expand the overall quantization range. Because the fixed step size of a uniform quantizer must cover the outliers, most activation values concentrated near the center are quantized with lower precision, resulting in significant quantization error. This issue stems directly from the non-uniformity of the activation distribution, and this mismatch inherently limits the performance of uniform quantizers. This empirical observation aligns with the theoretical limitations of uniform quantization, which we explore in the next paragraph.

Theoretical analysis. Given a input vector $\mathbf{x}$ with a probability density function $p(x)$, we aim to design a b-bit quantizer that maps $\mathbf{x}$ to $N = 2^b$ quantization points $\{q_k\}_{k=0}^{N-1}$. The input space is partitioned into N intervals $\{[a_k, a_{k+1}]\}_{k=0}^{N-1}$. The goal of the quantization is to minimize the mean squared error (MSE):

$$E = \sum_{k=0}^{N-1} \int_{a_k}^{a_{k+1}} (x - q_k)^2 p(x)\, dx. \quad (2)$$

To minimize Eq. (2), we differentiate it with respect to a_k and q_k, and solve for the optimal values. The resulting expressions satisfy the Lloyd-Max conditions [29] (Please refer to Appendix A for a detailed derivation):

(1) Decision boundary condition: The interval boundaries a_k are located at the midpoints between adjacent quantization points,

$$a_k = \frac{q_{k-1} + q_k}{2}. \quad (3)$$

(2) Centroid condition: The quantization points q_k are the conditional expectations of the corresponding intervals,

$$q_k = \frac{\int_{a_k}^{a_{k+1}} x p(x)\, dx}{\int_{a_k}^{a_{k+1}} p(x)\, dx}. \quad (4)$$

Obviously, the uniform quantizer naturally satisfies the decision boundary condition Eq. (3), because each decision boundary is the midpoint of adjacent quantization points. However, due to the non-uniformity of $p(x)$ within the quantization intervals, the optimal centroid of each interval (defined by Eq. (4)) may deviate from the midpoint, thus making uniform quantization no longer optimal.

Algorithm 1 Generate Householder orthogonal matrix

Input: Activation vector $\mathbf{x} \in \mathbb{R}^d$
Output: Householder orthogonal matrix $\mathbf{H} \in \mathbb{R}^{d\times d}$

$\gamma \leftarrow \|\mathbf{x}\|_2$ // Calculate input vector norm
$\mathbf{u} \leftarrow \mathcal{U}(-1,1)^d$ // Get uniform distribution in $[-1,1]^d$
$\mathbf{u} \leftarrow \gamma \cdot \mathbf{u}/\|\mathbf{u}\|_2$ // Match input vector norm
$\boldsymbol{\omega} \leftarrow \mathbf{x} - \mathbf{u}$ // Construct reflection axis
$\mathbf{H} \leftarrow \mathbf{I}_d - \frac{2}{\boldsymbol{\omega}^\top\boldsymbol{\omega}}\boldsymbol{\omega}\boldsymbol{\omega}^\top$ // Get the Householder matrix by Eq. (5)

Based on both empirical observation and theoretical analysis, our goal is to find a transformation F that maps $\mathbf{x}$ to a distribution that is closer to uniform. This transformation aims to align the quantization levels of the uniform quantizer with the optimal quantization points, thereby enabling the quantizer to approach the theoretical optimum characterized by Lloyd-Max quantization theory.

4 The Proposed Method

The RUQuant method comprises two key steps: uniformization of activation and weight distributions, and optimization of quantization error. In the first step, we utilize two types of orthogonal transformations—Householder reflections and Givens rotations—to map activations to a uniform distribution, thereby minimizing the loss introduced by uniform quantizers. In the second step, we design Learnable Householder matrices and fine-tune the orthogonal transformations by minimizing the discrepancy between model outputs before and after quantization, which improves the quantization accuracy. The overall framework of the RUQuant algorithm is illustrated in Figure 2.

4.1 Smoothing Activations and Weights

To uniformize activations and weights simultaneously, we consider the transformation: $\mathbf{Y} = \mathbf{W}\mathbf{A}^{-1}\mathbf{A}\mathbf{X}$, where computing the inverse can be costly. Therefore, we let $\mathbf{A}$ be an orthogonal matrix so that $\mathbf{A}^{-1} = \mathbf{A}^\top$, which reduces computational overhead and effectively altering the distribution. This form of equivalent transformation has been demonstrated effective in various prior works [11, 12]. In this section, we introduce a closed-form orthogonal matrix that maps activations into more uniformly distributed directions. Accordingly, the linear layer in LLMs can be equivalently rewritten as: $\mathbf{Y} = \mathbf{W}\mathbf{X} = \mathbf{W}\mathbf{Q_c}^\top\mathbf{Q_c}\mathbf{X}$, where $\mathbf{W} \in \mathbb{R}^{m\times d}$, $\mathbf{X} \in \mathbb{R}^{d\times N}$, $\mathbf{Q_c} \in \mathbb{R}^{d\times d}$, with d denoting the feature dimension and N the number of samples.

The uniformization step can be understood as follows: we uniformly sample target vectors and map an activation vector $\mathbf{x} \in \mathbb{R}^d$ to these target directions $\mathbf{u} \in \mathbb{R}^d$ in order to uniformize the distribution of activations. This mapping is realized through a composite orthogonal transformation composed of multiple Householder reflections and Givens rotations. To improve the expressive capacity and stability of the transformation, we perform multiple rounds of uniform sampling rather than a single one.

Specifically, a Householder reflection transformation $\mathbf{H} \in \mathbb{R}^{d\times d}$ can be uniquely determined by a vector $\boldsymbol{\omega} \in \mathbb{R}^d$:

$$\mathbf{H}(\boldsymbol{\omega}) = \mathbf{I}_d - \frac{2}{\boldsymbol{\omega}^\top\boldsymbol{\omega}}\boldsymbol{\omega}\boldsymbol{\omega}^\top. \tag{5}$$

Algorithm 2 Generate orthogonal matrix by Givens and permutations

Input: Activation vector $\mathbf{x} \in \mathbb{R}^d$, Number of permutations λ
Output: Orthogonal matrix $\mathbf{R} \in \mathbb{R}^{d\times d}$

$\mathbf{R} \leftarrow \mathbf{I}_d$ // Initialize the orthogonal matrix
$\mathbf{a} \leftarrow \mathbb{R}^{\frac{d}{2}}$ // Initialize angle vector
for $i = 1$ to λ **do**
  $\mathbf{P} \leftarrow$ Random permutation matrix
  $\mathbf{x} \leftarrow \mathbf{P}\mathbf{x}$
  $\mathbf{R} \leftarrow \mathbf{P}\mathbf{R}$ // Update the orthogonal matrix
  $\mathbf{u} \leftarrow \mathcal{U}(-1,1)^d$ // Get uniform distribution in $[-1,1]^d$
  for $j = 1$ to $n/2$ **do**
    $\mathbf{p} \leftarrow \mathbf{x}[2j : 2j+2]$ // Extract a sub-vector
    $\mathbf{v} \leftarrow \mathbf{u}[2j : 2j+2]$
    $\gamma \leftarrow \|\mathbf{p}\|_2$ // Calculate the norm of the sub-vector
    $\mathbf{v} \leftarrow \gamma \cdot \mathbf{v}/\|\mathbf{v}\|_2$ // Match sub-vector norm
    $\mathbf{a}_j \leftarrow$Calculate $\mathbf{a}_j$ by Eq. (8)
  end for
  $\mathbf{G}(\mathbf{a}) \leftarrow$ Calculate $\mathbf{G}$ by Eq. (7)
  $\mathbf{R} \leftarrow \mathbf{G}\mathbf{R}$ // Update the orthogonal matrix
end for

For any two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$ with $\|\mathbf{x}\|_2 = \|\mathbf{y}\|_2$ and $\mathbf{x} \neq \mathbf{y}$, there must exist a Householder matrix such that $\mathbf{y} = \mathbf{H}(\boldsymbol{\omega})\mathbf{x}$, where $\boldsymbol{\omega} = \mathbf{x} - \mathbf{y}$. Therefore, we can map each activation vector to a uniform vector $\mathbf{u}$ using a Householder matrix. The specific procedure is described in Algorithm 1.

In the 2D plane $\mathbb{R}^2$, a Givens rotation matrix can be uniquely determined by a single angle α:

$$\boldsymbol{\Gamma}(\alpha) = \begin{pmatrix} \cos\alpha & \sin\alpha \\ -\sin\alpha & \cos\alpha \end{pmatrix}. \tag{6}$$

Inspired by the RoPE (Rotary Position Embedding) method [30], we extend this two-dimensional rotation to higher dimensions by grouping the elements of a high-dimensional vector into pairs and rotating each pair separately. Thus, the rotation matrix $\mathbf{G} \in \mathbb{R}^{d\times d}$ can be uniquely determined by a vector $\mathbf{a} \in \mathbb{R}^{\frac{d}{2}}$:

$$\mathbf{G}(\mathbf{a}) = \text{BlockDiag}(\boldsymbol{\Gamma}(\mathbf{a}_1), \boldsymbol{\Gamma}(\mathbf{a}_2), \ldots, \boldsymbol{\Gamma}(\mathbf{a}_{\frac{d}{2}})). \tag{7}$$

For any two vectors $\mathbf{p}, \mathbf{v} \in \mathbb{R}^2$ with $\|\mathbf{p}\|_2 = \|\mathbf{v}\|_2$ and $\mathbf{p} \neq \mathbf{v}$, there must exist a Givens matrix such that $\mathbf{v} = \boldsymbol{\Gamma}(\mathbf{a}_i)\mathbf{p}$, and the rotation angle $\mathbf{a}_i$ can be computed as:

$$\cos\mathbf{a}_i = \frac{\mathbf{v}_1\mathbf{p}_1 + \mathbf{v}_2\mathbf{p}_2}{\mathbf{p}_1^2 + \mathbf{p}_2^2}, \quad \sin\mathbf{a}_i = \frac{\mathbf{v}_1\mathbf{p}_2 - \mathbf{v}_2\mathbf{p}_1}{\mathbf{p}_1^2 + \mathbf{p}_2^2}. \tag{8}$$

Since $\mathbf{G}$ is a block diagonal matrix with block size 2, it only smooths pairs of consecutive values, which are often already uniformly distributed. To address this, we introduce a random permutation matrix $\mathbf{P} \in \mathbb{R}^{d\times d}$ with $\mathbf{P}^\top\mathbf{P} = \mathbf{I}$ to shuffle the activation vector $\mathbf{x}$ before rotation. By alternating random permutations and block rotations, we form an orthogonal matrix $\mathbf{R}$. This process is repeated λ times. The specific procedure is described in Algorithm 2.

For each sampled activation vector $\mathbf{x}$, we compute a reflection orthogonal matrix and a rotation orthogonal matrix according to Algorithm 1 and Algorithm 2, respectively. To enhance stability, we adopt an iterative sampling approach to generate $\mathbf{Q}_c$. Let K denote

the number of sampling iterations. We perform K iterations of these two algorithms, resulting in

$$\mathbf{Q}_c^\top = (\mathbf{R}_1\mathbf{H}_1)^\top(\mathbf{R}_2\mathbf{H}_2)^\top \cdots (\mathbf{R}_K\mathbf{H}_K)^\top. \tag{9}$$

4.1.1 Smoothing Effect on Activation. Based on the method above, we can map a sampled activation vector to a uniformly distributed space. However, it is impractical to compute a dedicated projection matrix for every activation vector individually. Instead, we investigate whether the orthogonal projection matrix derived from a single sampled vector $\mathbf{x}$ from the activation matrix $\mathbf{X}$ can generalize to smooth the entire activation matrix. As shown in Theorem 1, the constructed orthogonal matrix is capable of effectively smoothing $\mathbf{X}$. The complete derivation is provided in Appendix B.

Theorem 1 (Smoothing effect of orthogonal transformation on activations). *Let $\mathbf{x} \in \mathbb{R}^d$ be an activation vector sampled from $\mathcal{N}(\boldsymbol{\mu}, \Sigma)$, and let $\mathbf{Q} \in \mathbb{R}^{d\times d}$ be an orthogonal matrix such that $\mathbf{Q}\mathbf{x} = \mathbf{u}$, where $\mathbf{u}$ is a fixed vector sampled from a uniform distribution. Then, for any new activation $\mathbf{x}_{new} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$, the transformed vector satisfies $\mathbf{Q}\mathbf{x}_{new} \sim \mathcal{N}(\mathbf{Q}\boldsymbol{\mu}, \mathbf{Q}\Sigma\mathbf{Q}^\top)$. Moreover, the transformation approximately satisfies*

$$\mathbf{Q}\boldsymbol{\mu} \approx \mathbf{u}, \quad \mathbb{E}[\mathbf{Q}\Sigma\mathbf{Q}^\top] = \frac{\mathrm{tr}(\Sigma)}{d}\mathbf{I}, \tag{10}$$

where $\mathbf{I}$ is the identity matrix.

We observe that the value of $\mathrm{tr}(\Sigma)/d$ is very small, indicating that the distribution of $\mathbf{x}$ after the transformation by the orthogonal matrix $\mathbf{Q}_c$ is centered around the uniform vector $\mathbf{u}$, with small isotropic fluctuations in each dimension. We show the mean vector and covariance matrix of $\mathbf{Q}_c\mathbf{X}$ in Figure 1. It can be seen that the mean of $\mathbf{Q}_c\mathbf{X}$ (Figure 1(b)) fluctuation range is very small, showing a relatively uniform trend, and the covariance matrix (Figure 1(d)) satisfies the isotropy as shown in Eq. (10). It can be seen that the transformed activations are concentrated around the mean vector and the fluctuation range is very small.

4.1.2 Smoothing Effect on Weight. The previous strategy focuses on smoothing the activation matrix $\mathbf{X}$ by applying an orthogonal transformation $\mathbf{Q}$. However, due to the equivalence of the transformation $\mathbf{W}\mathbf{X} = (\mathbf{W}\mathbf{Q}^\top)(\mathbf{Q}\mathbf{X})$, the same orthogonal matrix $\mathbf{Q}$ must be applied to the weight matrix $\mathbf{W}$ from the right. While $\mathbf{Q}$ is constructed solely based on the distribution of $\mathbf{X}$, we empirically observe that this transformation also has a smoothing effect on $\mathbf{W}$. Theoretically, the following theorem shows that applying the transpose of the activation-based orthogonal matrix $\mathbf{Q}$ to the weight matrix effectively reduces its distributional anisotropy. The full derivation is provided in Appendix C.

Theorem 2 (Smoothing effect of orthogonal transformation on weight). *Let $\mathbf{Q} \in \mathbb{R}^{d\times d}$ be an orthogonal matrix constructed from activation statistics (Section 4.1), and let a weight matrix $\mathbf{W} \in \mathbb{R}^{m\times d}$ be transformed as $\mathbf{W} \leftarrow \mathbf{W}\mathbf{Q}^\top$. Assume each row vector $\mathbf{w}^\top$ of $\mathbf{W}$ is independently drawn from $\mathcal{N}(\boldsymbol{\mu}_W, \Sigma_W)$, where $\boldsymbol{\mu}_W \in \mathbb{R}^d$ and $\Sigma_W \in \mathbb{R}^{d\times d}$. Then the transformed vector $\mathbf{Q}\mathbf{w}$ satisfies $\mathbf{Q}\mathbf{w} \sim \mathcal{N}(\mathbf{Q}\boldsymbol{\mu}_W, \mathbf{Q}\Sigma_W\mathbf{Q}^\top)$. Moreover, the transformation approximately satisfies*

$$\mathbf{Q}\boldsymbol{\mu}_W \approx \mathbf{0}, \quad \mathbb{E}[\mathbf{Q}\Sigma_W\mathbf{Q}^\top] = \frac{\mathrm{tr}(\Sigma_W)}{d}\mathbf{I}, \tag{11}$$

where $\mathbf{I}$ is the identity matrix.

In fact, our experimental observations also corroborate Eq (11): our $\mathbf{Q}_c$ not only effectively smooths the activations, but also smooths the weights. Although the original weights $\mathbf{W}$ are already relatively uniform, after the transformation by $\mathbf{W}\mathbf{Q}_c^\top$, the weight vectors become more evenly distributed and exhibit smaller variations across dimensions. Additional visualizations are provided in Appendix D.

4.1.3 Block-Wise Rotation. Although our method is very fast to directly construct the entire orthogonal matrix, there are two problems: 1) The time complexity of multiplying the orthogonal matrix $\mathbf{Q}_c \in \mathbb{R}^{d\times d}$ and the activation matrix $\mathbf{X} \in \mathbb{R}^{d\times N}$ is $O(d^2N)$, which will greatly increase the inference delay of the model. 2) The number of parameters of the orthogonal matrix $\mathbf{Q}_c$ is d^2, and storing $\mathbf{Q}_c$ will generate a huge memory overhead. Like DuQuant [12], we adopted the block-wise rotation and zigzag permutation strategy.

(1) **Block-Wise Processing**: We approximate the rotation matrix $\mathbf{Q}_c$ in a block manner. When the block size is B, we have:

$$\hat{\mathbf{Q}}_\mathbf{c} = \mathrm{BlockDiag}(\mathbf{Q}_{b_1}, \cdots, \mathbf{Q}_{b_{d/B}}), \tag{12}$$

where $\mathbf{Q}_{b_i} \in \mathbb{R}^{B\times B}$ is the shared square matrix for all blocks, i.e., $\mathbf{Q}_{b_i} = \mathbf{Q}_b$ for all i , and $\mathbf{Q}_b$ is constructed by Eq. (9). Therefore, the total number of parameters is reduced to B^2, and the computational complexity of multiplying the block-wise orthogonal matrix with the activation matrix is reduced to $O(BdN)$. In our experiments, we fix $B = 128$; for a weight matrix of size 4096×4096, this accounts for less than 0.1% of the original parameter count.

(2) **Zigzag Permutation**: Since the blocks are independent, zigzag permutation can reduce the variance between blocks[12]. Specifically, after the block-wise rotation, we introduce a permutation matrix $\mathbf{Z} \in \mathbb{R}^{d\times d}$ that satisfies $\mathbf{Z}\mathbf{Z}^\top = \mathbf{I}$. By shuffling the order of activation elements, the independence between blocks is disrupted, leading to a more uniform distribution across blocks and thereby improving the overall quantization performance.

Finally, the $\mathbf{Q}_c$ of the first step is: $\mathbf{Q}_c = \left(\hat{\mathbf{Q}}_c(1)\mathbf{Z}\hat{\mathbf{Q}}_c(2)\right)^\top$, where $\mathbf{Z}$ represents the permutation matrix generated by the zigzag permutation, and $\hat{\mathbf{Q}}_c(1), \hat{\mathbf{Q}}_c(2)$ represents the diagonal matrices of the first and second block-wise rotations.

4.2 Optimizing the Quantization Loss

In this section, to align the quantization loss of the uniform quantizer with the model loss and further improve quantization accuracy, we introduce a Learnable Householder matrix $\mathbf{Q}_l$. According to Eq. (5), a $d \times d$ reflection matrix $\mathbf{H}$ can be generated from a d-dimensional vector $\boldsymbol{\theta}$. Moreover, the matrix-vector multiplication can be computed efficiently as $\mathbf{H}\mathbf{X} = \mathbf{X} - \eta\boldsymbol{\theta}(\boldsymbol{\theta}^\top\mathbf{X})$, where $\eta = 2/(\boldsymbol{\theta}^\top\boldsymbol{\theta})$, resulting in a computational complexity of $O(dN)$. Additionally, the parameter count of $\boldsymbol{\theta}$ is only d for each activation. Therefore, introducing a Learnable Householder matrix incurs minimal parameter overhead and low computational cost. Specifically, after Step 1, we obtain $\mathbf{X}_{step1} = \mathbf{Q}_c\mathbf{X}$. Instead of dividing $\mathbf{X}_{step1}$ into blocks, we perform a single sampling to initialize $\boldsymbol{\theta}$ using Algorithm 1, and construct $\mathbf{Q}_l(\boldsymbol{\theta})$ via Eq. (5). We then fine-tune $\boldsymbol{\theta}$ by minimizing a block-wise loss designed to optimize the quantization

Table 1: The perplexity results (↓) of the LLaMA1, LLaMA2 and LLaMA3 model on the WikiText2 dataset.

#Bit	Method	1-7B	1-13B	1-30B	2-7B	2-13B	3-8B
FP16	-	5.68	5.09	4.10	5.47	4.88	6.14
W4A4	SmoothQuant	25.25	40.05	192.40	83.12	35.88	210.19
	OmniQuant	11.26	10.87	10.33	14.26	12.30	3.64e3
	QLLM	9.65	8.41	8.37	11.75	9.09	-
	DuQuant	6.40	5.65	4.72	6.28	5.42	8.56
	RUQuant	6.29	5.55	4.61	6.17	5.35	8.10
	RUQuant$_{\text{+fine-tune}}$	**6.14**	**5.53**	**4.55**	**6.04**	**5.29**	**7.99**
W6A6	SmoothQuant	25.25	40.05	192.40	83.12	35.88	7.07
	OmniQuant	11.26	10.87	10.33	14.26	12.30	7.24
	QLLM	9.65	8.41	8.37	11.75	9.09	-
	DuQuant	5.73	5.13	4.14	5.53	4.92	6.27
	RUQuant	**5.71**	**5.12**	**4.13**	**5.51**	**4.91**	**6.25**

accuracy:

$$\arg\min_{\theta} \left\| f(\mathbf{W}, \mathbf{X}) - f\left(\mathcal{Q}\left(\mathbf{W}\mathbf{Q}_c^\top \mathbf{Q}_l(\theta)^\top\right), \mathcal{Q}\left(\mathbf{Q}_l(\theta)\mathbf{Q}_c\mathbf{X}\right)\right) \right\|_{\mathrm{F}}^2, \tag{13}$$

where $f(\cdot)$ denotes a Transformer block, $\mathcal{Q}(\cdot)$ is the quantization function, and $\|\cdot\|_{\mathrm{F}}^2$ denotes the squared Frobenius norm.

4.3 The Overall Method

We provide a detailed algorithm in Appendix F. For each Transformer block $\mathcal{T}$, the procedure is performed in two main steps. In the first step, we apply Householder and Givens transformations to all linear layers within the block. The input $\mathbf{X}$ is first partitioned into blocks of size B. Then, all blocks share a common rotation matrix $\mathbf{Q}_c$, which, through the composite linear transformations of Householder reflections and Givens rotations, precisely maps the activations from a non-uniform to a uniform distribution. To further reduce inter-block variance, we apply a zigzag permutation followed by an additional rotation. In the second step, we initialize the parameters $\boldsymbol{\theta}$ of a Learnable Householder matrix using Algorithm 1, and fine-tune $\boldsymbol{\theta}$ by minimizing the difference in outputs of the Transformer block before and after quantization. This fine-tuning process helps mitigate the artifacts introduced by block-wise processing and better aligns the quantization loss with the overall model loss.

5 Experiment

5.1 Experimental Settings

To validate the superiority of our proposed method, we conduct two types of experiments: perplexity experiments and accuracy experiments. Following DuQuant [12], during the experimental procedure, each model does not see any task-specific data, and the accuracy experiments are also conducted in zero-shot setting.

Models and Datasets. For perplexity experiments, we evaluate several different models by quantizing the pre-trained LLMs: LLaMA1 (7B–30B) [31], LLaMA2 (7B–13B) [32], and LLaMA3 (8B), on the C4 [33] and WikiText2 [34] datasets. For zero-shot experiments, we measure the above-mentioned models on PIQA [35], ARC [36], HellaSwag [37], BoolQ [38], and WinoGrande [39] datasets. For the calibration dataset $\mathbf{X}$, we use 128 random selections of 2048-token segments from the WikiText2 dataset [34].

Baselines. We compare our method with several strong weight and activation quantization methods. (1) Smoothquant [10], (2) Omniquant [13], (3) QLLM [40], (4) DuQuant [12].

5.2 Implementation Details

Our main quantization levels are W4A4 and W6A6. We quantize all intermediate activations, with the exception of the Softmax output. Additionally, we apply a smooth pre-processing step to the model prior to quantization [10], which does not introduce any additional storage or inference latency.

We observe that nearly lossless compression can be achieved even without fine-tuning by Step 2. Therefore, we propose two schemes: RUQuant and RUQuant$_{\text{+fine-tune}}$. For RUQuant, we follow the two-step process, but do not fine-tune $\boldsymbol{\theta}$ in Step 2, which enables us to complete the quantization of a 13B parameter model in approximately one minute. For RUQuant$_{\text{+fine-tune}}$, we use gradient descent to fine-tune $\boldsymbol{\theta}$, which improves the accuracy but significantly increases the time cost. We only present RUQuant$_{\text{+fine-tune}}$ results under the W4A4 setting, as the performance without fine-tuning is already high for W6A6, making fine-tuning unnecessary in that case. In terms of quantization method, we adopted RTN quantization and clip the weights and activations. All quantization experiments are conducted on NVIDIA L20 GPUs. For more details and specific hyperparameters, please refer to Appendix E.

5.3 Analysis of Perplexity Results

Table 1 reports perplexity results of various quantization methods on LLaMA1, LLaMA2, and LLaMA3 using the WikiText2 dataset. The proposed RUQuant method consistently outperforms strong baselines like SmoothQuant, OmniQuant, QLLM, and DuQuant under both W4A4 and W6A6 settings, often matching full-precision performance. Overall, RUQuant provides effective compression with minimal perplexity degradation across model scales and generations, demonstrating strong robustness and generalizability. On the C4 dataset, RUQuant still demonstrate superior performance, please refer to the Appendix G for detailed results.

Table 2: Zero-shot QA (↑) results of LLaMA2 and LLaMA3 models under 4-bit weight-activation quantization.

Model	Method	PIQA	ARC-E	ARC-C	BoolQ	HellaSwag	WinoGrande	Avg.
LLaMA2-7B W4A4	FP16	76.88	53.54	40.53	71.13	72.96	67.25	63.72
	SmoothQuant	60.17	35.23	27.13	57.92	37.08	49.57	44.52
	OmniQuant	65.61	44.28	30.38	62.66	53.51	51.85	51.38
	QLLM	67.68	44.40	30.89	-	58.45	56.59	51.60
	DuQuant	75.24	51.89	36.77	67.86	69.54	62.12	60.57
	RUQuant	**75.84**	**52.82**	**39.51**	**69.57**	**70.06**	**63.61**	**61.90**
	RUQuant$_{+\text{fine-tune}}$	74.65	51.18	38.65	68.59	69.65	61.72	60.74
LLaMA2-13B W4A4	FP16	79.05	57.91	44.20	69.02	76.60	69.69	66.08
	SmoothQuant	62.30	40.28	30.72	60.49	42.24	49.96	47.67
	OmniQuant	69.80	47.22	33.79	65.47	59.34	55.49	55.19
	QLLM	70.46	48.48	34.39	-	62.80	55.41	54.31
	DuQuant	77.31	55.60	41.55	66.61	73.68	66.06	63.47
	RUQuant	78.24	55.47	41.30	68.20	**74.07**	**67.64**	**64.15**
	RUQuant$_{+\text{fine-tune}}$	**78.51**	**56.14**	**41.98**	**68.41**	73.78	65.98	64.13
LLaMA3-8B W4A4	FP16	80.85	77.78	53.41	81.28	79.16	72.84	74.22
	SmoothQuant	54.57	31.9	24.23	52.72	31.26	51.14	40.97
	OmniQuant	50.22	26.94	24.57	37.98	26.55	50.20	36.08
	DuQuant	75.68	68.48	41.81	71.99	73.07	66.22	66.21
	RUQuant	**76.93**	**70.45**	**44.97**	**75.20**	73.36	66.14	**67.84**
	RUQuant$_{+\text{fine-tune}}$	76.12	69.53	42.92	71.93	**74.13**	**67.01**	66.94

Table 3: Zero-shot QA (↑) results of LLaMA2 and LLaMA3 models under 6-bit weight-activation quantization.

Model	Method	PIQA	ARC-E	ARC-C	BoolQ	HellaSwag	WinoGrande	Avg.
LLaMA2-7B W6A6	FP16	76.88	53.54	40.53	71.13	72.96	67.25	63.72
	SmoothQuant	75.57	53.62	39.93	69.54	71.76	66.14	62.76
	OmniQuant	76.55	53.83	40.96	68.75	55.89	65.59	60.26
	QLLM	77.48	52.99	39.33	-	71.38	65.98	61.43
	DuQuant	76.99	**52.99**	**40.87**	**70.40**	72.49	67.32	63.51
	RUQuant	**77.04**	52.97	40.61	**70.40**	**72.71**	**67.88**	**63.60**
LLaMA2-13B W6A6	FP16	79.05	57.91	44.20	69.02	76.60	69.69	66.08
	SmoothQuant	78.29	57.41	43.86	69.50	75.02	66.93	65.17
	OmniQuant	78.24	57.58	43.86	71.10	75.52	68.35	65.78
	QLLM	78.78	58.29	43.77	-	75.10	68.43	64.87
	DuQuant	78.62	56.94	43.43	**68.35**	76.19	69.22	65.46
	RUQuant	**79.16**	**57.74**	**44.45**	67.83	**76.45**	**69.77**	**65.90**
LLaMA3-8B W6A6	FP16	80.85	77.78	53.41	81.28	79.16	72.84	74.22
	SmoothQuant	78.94	75.88	49.49	77.58	77.39	70.80	71.68
	OmniQuant	78.90	73.95	47.35	74.95	76.77	70.56	70.41
	DuQuant	80.20	77.27	52.05	80.12	**79.14**	72.77	73.59
	RUQuant	**80.25**	**77.44**	**52.73**	**80.40**	78.99	**73.24**	**73.84**

5.4 Analysis of Zero-Shot Results

Tables 2 and 3 report zero-shot QA results on LLaMA2-7B, LLaMA2-13B, and LLaMA3-8B under 4-bit and 6-bit quantization. RUQuant consistently outperforms strong baselines such as SmoothQuant, OmniQuant, QLLM, and DuQuant. Under 4-bit quantization, RUQuant shows clear accuracy improvements over all baselines across all models. In the 6-bit setting, it remains competitive and closely matches full-precision performance. Overall, RUQuant demonstrates

Table 4: Influence of different components in RUQuant under 4-bit weight-activation quantization.

Method	LLaMA2-7B		LLaMA2-13B	
	WikiText2 ↓	C4 ↓	WikiText2 ↓	C4 ↓
w/o Householder	33.04e3	27.34e3	NaN	NaN
w/o Givens	27.42	35.20	1.32e3	1.54e3
w/o LH	6.18	7.64	5.36	6.88
RUQuant	**6.17**	**7.63**	**5.35**	**6.87**

Table 5: Running time (in seconds) of RUQuant and DuQuant on LLaMA1 models with W4A4 quantization on WikiText2.

Method	LLaMA1-7B	LLaMA1-13B	LLaMA1-30B
DuQuant	44	66	136
RUQuant	47	68	145

robust and generalizable performance across different models and quantization levels, particularly excelling in the 4-bit scenarios.

5.5 Ablation Study

5.5.1 Influence of Different Components. We evaluate the impact of three key components on compression performance: (1) removing the Householder reflection step, (2) removing the Givens rotation step, and (3) removing the entire Learnable Householder matrix (LH). Table 4 shows the results under 4-bit weight-activation quantization on LLaMA2-7B and LLaMA2-13B. As observed, removing the Householder reflection step (w/o Householder) leads to the most significant performance degradation. In this setting, the model nearly loses its representational capacity, and fails to converge (NaN) on LLaMA2-13B. In contrast, removing the Givens rotation (w/o Givens) also causes a performance drop, but to a lesser extent. When the entire Learnable Householder matrix is removed (w/o LH), performance slightly decreases but remains close to that of the full method (RUQuant), indicating that LH can be used in precision-sensitive scenarios.

5.5.2 Runtime. Table 5 presents the runtime required to compress the LLaMA models of sizes 7B, 13B, and 30B. As shown, RUQuant takes only around 68 seconds to compress the 13B model, demonstrating the ability to complete compression within approximately one minute. This makes RUQuant competitive with DuQuant in terms of runtime performance. Furthermore, RUQuant shows consistent performance across all model scales, indicating good scalability and strong potential for practical deployment.

5.6 Inference Speedup

To assess the layer-wise inference speedup enabled by our RUQuant method, we follow the same evaluation protocol and use the W4A4 kernel as described in [25]. Table 6 reports the speedup in the prefill stage for LLaMA2-7B and LLaMA2-13B, measured on a single NVIDIA RTX 3090 GPU with a prefill sequence length of 2048. Across all batch sizes, RUQuant consistently delivers competitive acceleration, achieving up to 1.935× speedup for LLaMA2-7B and 2.104× for LLaMA2-13B. Notably, if we allow for a slight trade-off in performance by removing the Learnable Householder (w/o LH), the perplexity of the model increases by only 0.01 (as shown in Table 4), while still outperforming DuQuant in accuracy. Moreover, the inference speedup becomes identical to that of DuQuant, making this configuration highly practical in latency-sensitive scenarios.

Table 6: Layer-wise speedup during pre-filling stage for W4A4 quantization.

Model	Batch Size	RUQuant	w/o LH
LLaMA2-7B	1	1.816×	1.934×
	4	1.845×	1.993×
	16	1.935×	2.104×
LLaMA2-13B	1	1.934×	2.154×
	4	1.993×	2.293×
	16	2.104×	2.474×

Table 7: Peak memory usage of the proposed method compared with DuQuant and FP16 during the pre-filling and decoding stages across varying batch sizes.

Batch Size	Method	Pre-filling(GB)	Decoding(GB)
1	FP16	14.029	13.702
	DuQuant	4.287	3.922
	RUQuant	4.287	3.922
4	FP16	18.212	16.939
	DuQuant	6.228	4.768
	RUQuant	6.228	4.768

5.7 Memory Consumption

We evaluate the peak memory usage of RUQuant under the same settings as DuQuant, using a W4A4 kernel on LLaMA2-7B and a single NVIDIA RTX 3090 GPU. We process 2048 tokens during the pre-filling stage and run 128 decoding steps. Results are shown in Table 7. Compared to the FP16 baseline, RUQuant achieves up to 3.3× and 3.5× memory reduction for pre-filling and decoding respectively at batch size 1, and 2.9× and 3.6× at batch size 4. RUQuant matches DuQuant in both pre-filling and decoding stages across all batch sizes, demonstrating equivalent memory efficiency.

6 Conclusion

We propose RUQuant, a theoretically grounded, two-stage orthogonal transformation method for post-training activation quantization in large language models. By aligning non-uniform activation distributions with uniform quantizers through structured orthogonal mappings, RUQuant effectively minimizes quantization error without requiring model fine-tuning. Empirical results on a 13B model show that RUQuant achieves up to 99.8% of full-precision accuracy with W6A6 and 97% with W4A4 quantization in approximately one minute. In future work, we will extend the theoretical framework of RUQuant to codebook-based quantization, pursuing more aggressive memory compression.

7 Acknowledgements

This work was supported by National Natural Science Foundation of China (No. 62206038, 62106035), the Strategic Priority Research Program of the Chinese Academy of Sciences (No. XDA0490301), Liaoning Binhai Laboratory Project (No. LBLF-2023-01), and Xiaomi Young Talents Program.

References

[1] Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, Matthias Gallé, et al. BLOOM: A 176b-parameter open-access multilingual language model. *CoRR*, abs/2211.05100, 2022.

[2] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1, 2019.

[3] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. In *North American Chapter of the Association for Computational Linguistics (NAACL)*, pages 4171–4186, 2019.

[4] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized BERT pretraining approach. *CoRR*, abs/1907.11692, 2019.

[5] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Conference on Neural Information Processing Systems (NeurIPS)*, pages 5998–6008, 2017.

[6] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. GPTQ: accurate post-training quantization for generative pre-trained transformers. *CoRR*, abs/2210.17323, 2022.

[7] Tim Dettmers, Ruslan Svirschevski, Vage Egiazarian, Denis Kuznedelev, Elias Frantar, Saleh Ashkboos, Alexander Borzunov, Torsten Hoefler, and Dan Alistarh. Spqr: A sparse-quantized representation for near-lossless LLM weight compression. In *The Twelfth International Conference on Learning Representations(ICLR)*, 2024.

[8] Jerry Chee, Yaohui Cai, Volodymyr Kuleshov, and Christopher De Sa. Quip: 2-bit quantization of large language models with guarantees. In *Conference on Neural Information Processing Systems (NeurIPS)*, 2023.

[9] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang, and Song Han. AWQ: activation-aware weight quantization for LLM compression and acceleration. *CoRR*, abs/2306.00978, 2023.

[10] Guangxuan Xiao, Ji Lin, Mickaël Seznec, Hao Wu, Julien Demouth, and Song Han. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning (ICML)*, pages 38087–38099, 2023.

[11] Yuexiao Ma, Huixia Li, Xiawu Zheng, Feng Ling, Xuefeng Xiao, Rui Wang, Shilei Wen, Fei Chao, and Rongrong Ji. Affinequant: Affine transformation quantization for large language models. In *The Twelfth International Conference on Learning Representations(ICLR)*, 2024.

[12] Haokun Lin, Haobo Xu, Yichen Wu, Jingzhi Cui, Yingtao Zhang, Linzhan Mou, Linqi Song, Zhenan Sun, and Ying Wei. Duquant: Distributing outliers via dual transformation makes stronger quantized llms. In *Conference on Neural Information Processing Systems (NeurIPS)*, 2024.

[13] Wenqi Shao, Mengzhao Chen, Zhaoyang Zhang, Peng Xu, Lirui Zhao, Zhiqian Li, Kaipeng Zhang, Peng Gao, Yu Qiao, and Ping Luo. Omniquant: Omnidirectionally calibrated quantization for large language models. In *The Twelfth International Conference on Learning Representations(ICLR)*, 2024.

[14] Yuxuan Sun, Ruikang Liu, Haoli Bai, Han Bao, Kang Zhao, Yuening Li, Jiaxin Hu, Xianzhi Yu, Lu Hou, Chun Yuan, Xin Jiang, Wulong Liu, and Jun Yao. Flatquant: Flatness matters for LLM quantization. *CoRR*, abs/2410.09426, 2024.

[15] Zechun Liu, Changsheng Zhao, Igor Fedorov, Bilge Soran, Dhruv Choudhary, Raghuraman Krishnamoorthi, Vikas Chandra, Yuandong Tian, and Tijmen Blankevoort. Spinquant: LLM quantization with learned rotations. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.

[16] Xunyu Zhu, Jian Li, Yong Liu, Can Ma, and Weiping Wang. A survey on model compression for large language models. *CoRR*, abs/2308.07633, 2023.

[17] Zhuocheng Gong, Jiahao Liu, Jingang Wang, Xunliang Cai, Dongyan Zhao, and Rui Yan. What makes quantization for large language models hard? an empirical study from the lens of perturbation. *CoRR*, 2024.

[18] Markus Nagel, Rana Ali Amjad, Mart Van Baalen, Christos Louizos, and Tijmen Blankevoort. Up or down? adaptive rounding for post-training quantization. In *International Conference on Machine Learning*, pages 7197–7206. PMLR, 2020.

[19] Junyi Wu, Haoxuan Wang, Yuzhang Shang, Mubarak Shah, and Yan Yan. Ptq4dit: Post-training quantization for diffusion transformers. *arXiv preprint arXiv:2405.16005*, 2024.

[20] Tianchen Zhao, Xuefei Ning, Tongcheng Fang, Enshu Liu, Guyue Huang, Zinan Lin, Shengen Yan, Guohao Dai, and Yu Wang. Mixdq: Memory-efficient few-step text-to-image diffusion models with metric-decoupled mixed precision quantization. *arXiv preprint arXiv:2405.17873*, 2024.

[21] Han Liu, Haotian Gao, Xiaotong Zhang, Changya Li, Feng Zhang, Wei Wang, Fenglong Ma, and Hong Yu. SEPTQ: A simple and effective post-training quantization paradigm for large language models. In *Conference on Knowledge Discovery and Data Mining(KDD)*, pages 812–823, 2025.

[22] Yuhang Li, Ruokai Yin, Donghyun Lee, Shiting Xiao, and Priyadarshini Panda. GPTAQ: efficient finetuning-free quantization for asymmetric calibration. In *Forty-second International Conference on Machine Learning(ICML)*, 2025.

[23] Tianyi Zhang and Anshumali Shrivastava. Leanquant: Accurate and scalable large language model quantization with loss-error-aware grid. In *The Thirteenth International Conference on Learning Representations(ICLR)*, 2025.

[24] Zechun Liu, Barlas Oguz, Changsheng Zhao, Ernie Chang, Pierre Stock, Yashar Mehdad, Yangyang Shi, Raghuraman Krishnamoorthi, and Vikas Chandra. LLM-QAT: data-free quantization aware training for large language models. In *Findings of the Association for Computational Linguistics(ACL)*, 2024.

[25] Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L. Croci, Bo Li, Pashmina Cameron, Martin Jaggi, Dan Alistarh, Torsten Hoefler, and James Hensman. Quarot: Outlier-free 4-bit inference in rotated llms. In *Conference on Neural Information Processing Systems (NeurIPS)*, 2024.

[26] Jingyang Xiang and Sai Qian Zhang. Dfrot: Achieving outlier-free and massive activation-free for rotated llms with refined rotation. *CoRR*, abs/2412.00648, 2024.

[27] Eranga Bandara, Peter Foytik, Sachin Shetty, Ravi Mukkamala, Abdul Rahman, Xueping Liang, Wee Keong Ng, and Kasun De Zoysa. Slicegpt - openai GPT-3.5 llm, blockchain and non-fungible token enabled intelligent 5g/6g network slice broker and marketplace. In *21st IEEE Consumer Communications & Networking Conference, CCNC 2024, Las Vegas, NV, USA, January 6-9, 2024*, 2024.

[28] Xin Wang, Yu Zheng, Zhongwei Wan, and Mi Zhang. SVD-LLM: truncation-aware singular value decomposition for large language model compression. *CoRR*, abs/2403.07378, 2024.

[29] S. M. Borodkin, A. M. Borodkin, and I. B. Muchnik. Optimal requantization of deep grayscale images and lloyd-max quantization. *IEEE Trans. Image Process.*, 15, 2006.

[30] Jianlin Su, Murtadha H. M. Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568, 2024.

[31] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[32] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[33] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.*, 21, 2020.

[34] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations (ICLR)*, 2017.

[35] Sandeep Tata and Jignesh M. Patel. Piqa: An algebra for querying protein data sets. In *International Conference on Statistical and Scientific Database Management (SSDBM)*, pages 141–150, 2003.

[36] Sumithra Bhakthavatsalam, Daniel Khashabi, Tushar Khot, Bhavana Dalvi Mishra, Kyle Richardson, Ashish Sabharwal, Carissa Schoenick, Oyvind Tafjord, and Peter Clark. Think you have solved direct-answer question answering? try arc-da, the direct-answer AI2 reasoning challenge. *CoRR*, abs/2102.03315, 2021.

[37] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 4791–4800, 2019.

[38] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. Boolq: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 2924–2936, 2019.

[39] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: an adversarial winograd schema challenge at scale. *Commun. ACM*, 64, 2021.

[40] Jing Liu, Ruihao Gong, Xiuying Wei, Zhiwei Dong, Jianfei Cai, and Bohan Zhuang. QLLM: accurate and efficient low-bitwidth quantization for large language models. In *The Twelfth International Conference on Learning Representations(ICLR)*, 2024.

[41] Ron Banner, Yury Nahshan, and Daniel Soudry. Post training 4-bit quantization of convolutional networks for rapid-deployment. In *Conference on Neural Information Processing Systems (NeurIPS)*, pages 7948–7956, 2019.

A The Detailed Proof Procedure of Lloyd-Max Conditions

Here we provide the proof procedure for Eq. (3) and Eq. (4). Let $\mathbf{x}$ be an input vector with probability density function $p(x)$. A b-bit Lloyd-Max quantizer aims to map $\mathbf{x}$ to $N = 2^b$ quantization levels $\{q_k\}_{k=0}^{N-1}$. The input space is partitioned into N intervals $\{[a_k, a_{k+1}]\}_{k=0}^{N-1}$. The quantizer is designed to minimize the mean squared error (MSE) between the input and its quantized representation, defined as:

$$E = \sum_{k=0}^{N-1} \int_{a_k}^{a_{k+1}} (x - q_k)^2 p(x)\, dx. \tag{14}$$

To find the optimal quantization points q_k, we take the partial derivative of E with respect to q_k and set it to zero:

$$\frac{\partial E}{\partial q_k} = 0. \tag{15}$$

Since the error E is a sum of integrals, and only the k-th integral depends on q_k, we have:

$$\frac{\partial E}{\partial q_k} = \frac{\partial}{\partial q_k} \int_{a_k}^{a_{k+1}} (x - q_k)^2 p(x)\, dx. \tag{16}$$

Using the chain rule, we obtain:

$$\frac{\partial E}{\partial q_k} = \int_{a_k}^{a_{k+1}} \frac{\partial}{\partial q_k} (x-q_k)^2 p(x)\, dx = \int_{a_k}^{a_{k+1}} -2(x-q_k)p(x)\, dx = 0. \tag{17}$$

This simplifies to:

$$\int_{a_k}^{a_{k+1}} xp(x)\, dx = q_k \int_{a_k}^{a_{k+1}} p(x)\, dx. \tag{18}$$

Thus, the optimal quantization point q_k is the centroid of the interval $[a_k, a_{k+1}]$:

$$q_k = \frac{\int_{a_k}^{a_{k+1}} xp(x)\, dx}{\int_{a_k}^{a_{k+1}} p(x)\, dx}. \tag{19}$$

Next, we minimize E with respect to the decision boundaries a_k. The partial derivative of E with respect to a_k is:

$$\frac{\partial E}{\partial a_k} = 0. \tag{20}$$

Since a_k affects both the $(k-1)$-th and k-th integrals, we have:

$$\frac{\partial E}{\partial a_k} = \frac{\partial}{\partial a_k} \left(\int_{a_{k-1}}^{a_k} (x - q_{k-1})^2 p(x)\, dx + \int_{a_k}^{a_{k+1}} (x - q_k)^2 p(x)\, dx \right). \tag{21}$$

Applying the Leibniz integral rule, the derivative is evaluated at a_k:

$$\frac{\partial E}{\partial a_k} = (a_k - q_{k-1})^2 p(a_k) - (a_k - q_k)^2 p(a_k) = 0. \tag{22}$$

This implies:

$$|a_k - q_{k-1}| = |a_k - q_k|. \tag{23}$$

Thus, the optimal decision boundary a_k is the midpoint between the adjacent quantization points:

$$a_k = \frac{q_{k-1} + q_k}{2}. \tag{24}$$

In summary, the Lloyd-Max conditions for optimal quantization are:

$$q_k = \frac{\int_{a_k}^{a_{k+1}} xp(x)\, dx}{\int_{a_k}^{a_{k+1}} p(x)\, dx}. \tag{25}$$

and

$$a_k = \frac{q_{k-1} + q_k}{2}. \tag{26}$$

B The Detailed Proof Procedure of Theorem 1

Here we provide the proof procedure for Theorem 1. We observe that the activation $\mathbf{X}$ follow a multivariate normal distribution $\mathcal{N}(\boldsymbol{\mu}, \Sigma)$, where $\boldsymbol{\mu} \in \mathbb{R}^d$ is the mean vector and $\Sigma \in \mathbb{R}^{d\times d}$ is the covariance matrix. Moreover, it is assumed that $\|\boldsymbol{\mu}\|_2^2 \gg \mathrm{tr}(\Sigma)$, which aligns with previous findings [9, 10] that the variance within rows of $\mathbf{X}$ is small, while the variance between rows is large.

Given a sampled vector $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$ and a uniformly distributed vector $\hat{\mathbf{u}} \sim \mathbf{U}(-1, 1)$, we define $\mathbf{u} = \frac{\|\mathbf{x}\|_2}{\|\hat{\mathbf{u}}\|_2}\hat{\mathbf{u}}$. We then determine a unique orthogonal matrix $\mathbf{Q}$ (computed via Householder or Givens transformations) such that $\mathbf{u} = \mathbf{Q}\mathbf{x}$. For another sample $\mathbf{x}_{\text{new}} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$ from $\mathbf{X}$ with the same distribution as $\mathbf{x}$, considering the linear transformation of Gaussian vectors, we have:

$$\mathbf{Q}\mathbf{x}_{\text{new}} \sim \mathcal{N}(\mathbf{Q}\boldsymbol{\mu}, \mathbf{Q}\Sigma\mathbf{Q}^T). \tag{27}$$

Given any permutation matrix $\mathbf{P}$ and diagonal sign matrix $\mathbf{D}$:

$$\mathbf{PDu} \overset{d}{=} \mathbf{u}, \tag{28}$$

and considering $\mathbf{Qx} = \mathbf{u}$, we obtain:

$$\mathbf{PDQx} \overset{d}{=} \mathbf{Qx}. \tag{29}$$

Since $\mathbf{x}$ is arbitrary, we have:

$$\mathbf{PDQ} \overset{d}{=} \mathbf{Q}. \tag{30}$$

Thus,

$$\mathbb{E}_{\mathbf{Q}}[\mathbf{Q}\Sigma\mathbf{Q}^T] = \mathbb{E}_{\mathbf{Q}}[\mathbf{PDQ}\Sigma\mathbf{Q}^T\mathbf{DP}^T] = \mathbf{PD}\mathbb{E}_{\mathbf{Q}}[\mathbf{Q}\Sigma\mathbf{Q}^T]\mathbf{DP}^T. \tag{31}$$

Considering the algebra generated by the symmetric group $\{\mathbf{PD}\}$, which is irreducible in $\mathbb{R}^d$, any matrix commuting with all $\mathbf{PD}$ must be a scalar matrix $c\mathbf{I}$. Taking the trace operation, we have:

$$\mathrm{tr}(c\mathbf{I}) = cd = \mathrm{tr}(\mathbb{E}_{\mathbf{Q}}[\mathbf{Q}\Sigma\mathbf{Q}^T]) = \mathbb{E}_{\mathbf{Q}}[\mathrm{tr}(\Sigma)] = \mathrm{tr}(\Sigma). \tag{32}$$

Solving for c, we obtain:

$$c = \frac{\mathrm{tr}(\Sigma)}{d}. \tag{33}$$

Thus, we have:

$$\mathbb{E}_{\mathbf{Q}}[\mathbf{Q}\Sigma\mathbf{Q}^T] = \frac{\mathrm{tr}(\Sigma)}{d}\mathbf{I}. \tag{34}$$

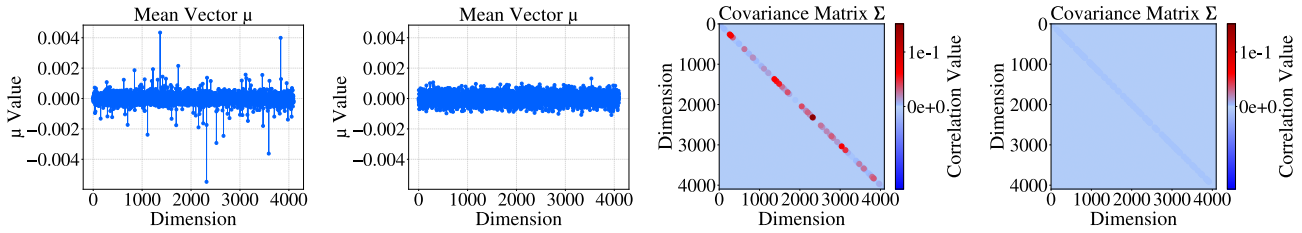
Given that the diagonal elements of the activation covariance matrix are small, we conclude that the covariance matrix is:

$$\mathbb{E}_{\mathbf{Q}}[\mathbf{Q}\Sigma\mathbf{Q}^T] = \lambda\mathbf{I}, \quad \lambda \approx 0, \tag{35}$$

indicating that the transformed covariance expectation is isotropic noise.

Let $\mathbf{x} = \boldsymbol{\mu} + \epsilon$, where $\epsilon \sim \mathcal{N}(0, \Sigma)$. Since $\mathbf{u} = \mathbf{Qx} = \mathbf{Q}\boldsymbol{\mu} + \mathbf{Q}\epsilon$, we have:

$$\mathbf{Q}\boldsymbol{\mu} = \mathbf{u} - \mathbf{Q}\epsilon. \tag{36}$$



(a) Original (b) RUQuant processed (c) Original (d) RUQuant processed

Figure 3: The mean and covariance visualizations of weights before and after RUQuant processing. (a) Original mean vector. (b) Mean vector after RUQuant processing. (c) Original covariance matrix. (d) Covariance matrix after RUQuant processing.

Given $\|\mathbf{x}\|_2^2 = \|\boldsymbol{\mu}\|_2^2 + 2\boldsymbol{\mu}^T\epsilon + \|\epsilon\|_2^2$, and knowing that $\epsilon \sim \mathcal{N}(0, \Sigma)$, $\mathbb{E}[\boldsymbol{\mu}^T\epsilon] = 0$, and $\mathbb{E}[\|\epsilon\|_2^2] = \mathrm{tr}(\Sigma)$, we obtain:

$$\mathbb{E}[\|\mathbf{x}\|_2^2] = \|\boldsymbol{\mu}\|_2^2 + \mathrm{tr}(\Sigma). \tag{37}$$

Since $\|\boldsymbol{\mu}\|_2^2 \gg \mathrm{tr}(\Sigma)$, we have:

$$\|\mathbf{u}\|_2^2 = \|\mathbf{x}\|_2^2 \approx \|\boldsymbol{\mu}\|_2^2 = \|\mathbf{Q}\boldsymbol{\mu}\|_2^2. \tag{38}$$

Moreover,

$$\cos\langle \mathbf{u}, \mathbf{Q}\boldsymbol{\mu}\rangle = \frac{\mathbf{u}^\top \cdot \mathbf{Q}\boldsymbol{\mu}}{\|\mathbf{u}\|_2 \cdot \|\mathbf{Q}\boldsymbol{\mu}\|_2} = \frac{\mathbf{u}^\top \cdot \mathbf{Q}\boldsymbol{\mu}}{\|\mathbf{x}\|_2 \cdot \|\boldsymbol{\mu}\|_2}. \tag{39}$$

Thus,

$$\mathbb{E}[\cos\langle \mathbf{x}_u, \mathbf{Q}\boldsymbol{\mu}\rangle] = \mathbb{E}\left[\frac{\|\boldsymbol{\mu}\|_2^2 + \boldsymbol{\mu}^\top\epsilon}{\|\mathbf{x}\|_2 \cdot \|\boldsymbol{\mu}\|_2}\right] \approx 1. \tag{40}$$

Therefore,

$$\mathbf{Q}\boldsymbol{\mu} \approx \mathbf{u}. \tag{41}$$

In summary, the transformed $\mathbf{X}$ is distributed around the uniform vector $\mathbf{u}$, with each dimension exhibiting small isotropic fluctuations.

C The Detailed Proof Procedure of Theorem 2

Here we provide the proof procedure for Theorem 2. Owing to the equivalent transformation, the weight matrix $\mathbf{W}$ in a linear layer of a large-scale model is subject to the following transformation:

$$\mathbf{W} \leftarrow \mathbf{W}\mathbf{Q}^\top. \tag{42}$$

The row vectors $\mathbf{w}^\top$ of $\mathbf{W}$ follow a multivariate normal distribution [41] $\mathcal{N}(\boldsymbol{\mu}_W, \Sigma_W)$, where $\boldsymbol{\mu}_W \in \mathbb{R}^d$ is the mean vector and $\Sigma_W \in \mathbb{R}^{d\times d}$ is the covariance matrix, we observe that the distribution of $\mathbf{W}$ is relatively uniform, with the mean vector $\boldsymbol{\mu}_W \approx \mathbf{0}$. Therefore:

$$\mathbf{Q}\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}\Sigma_W\mathbf{Q}^T). \tag{43}$$

The covariance matrix after transformation still satisfies:

$$\mathbb{E}_\mathbf{Q}[\mathbf{Q}\Sigma_W\mathbf{Q}^T] = \frac{\mathrm{tr}(\Sigma_W)}{d}\mathbf{I}. \tag{44}$$

Thus, the covariance dominates the shape of the distribution, and the expected covariance is isotropic noise. Consequently, the transformed $\mathbf{w}^\top$ is expected to be distributed isotropically, and $\mathbf{Q}$ also serves to smooth $\mathbf{W}$.

Algorithm 3 RUQuant quantization algorithm

Input: Transfomer Block $\mathcal{T}$, Block Size B, Epoch $\mathcal{E}$, Number of Random Permutations λ, Number of Sampling Times K, Zigzag Permutation Times T.
Output: $Q(\mathbf{W}), Q(\mathbf{X})$

for layer in $\mathcal{T}$ **do**
  $\mathbf{X} \leftarrow$ layer.input
  $\mathbf{W} \leftarrow$ layer.weight
  for $i = 1$ to T **do**
    $\mathbf{X}, \mathbf{W} \leftarrow \mathrm{Rotation}(\mathbf{X}, \mathbf{W}, B, K, \lambda)$
    $\mathbf{X}, \mathbf{Z} \leftarrow$Zigzag_Permutation($\mathbf{X}$)
    $\mathbf{W} \leftarrow \mathbf{W}\mathbf{Z}^\top$
  end for
  $\mathbf{X}, \mathbf{W} \leftarrow \mathrm{Rotation}(\mathbf{X}, \mathbf{W}, B, K, \lambda)$
  $\mathbf{x} \leftarrow \mathrm{Sample}(\mathbf{X})$
  $\mathbf{u} \leftarrow \mathcal{U}(-1, 1)^n$
  $\mathbf{u} \leftarrow \mathbf{u}(\|\mathbf{x}\|/\|\mathbf{u}\|)$
  $\boldsymbol{\theta} \leftarrow \mathbf{x} - \mathbf{u}$
  $\mathbf{X} \leftarrow \mathbf{X} - (2/\boldsymbol{\theta}^\top\boldsymbol{\theta})\boldsymbol{\theta}(\boldsymbol{\theta}^\top\mathbf{X})$
  $\mathbf{W} \leftarrow \mathbf{W} - (2/\boldsymbol{\theta}^\top\boldsymbol{\theta})\mathbf{W}\boldsymbol{\theta}\boldsymbol{\theta}^\top$
end for
for $j = 1$ to $\mathcal{E}$ **do**
  $L \leftarrow$ objective function from Eq. (13)
  $\boldsymbol{\theta} \leftarrow \mathrm{adam}(\boldsymbol{\theta}, \frac{\partial L}{\partial \boldsymbol{\theta}})$
end for
$\mathbf{W} \leftarrow Q(\mathbf{W})$
$\mathbf{X} \leftarrow Q(\mathbf{X})$

D Visual Analysis of W and $\mathbf{W}\mathbf{Q}_c^\top$

Figure 3 shows the effect of the rotation matrix on $\mathbf{W}$. It can be found that the rotation matrix solved according to the characteristics of $\mathbf{X}$ can also have a smoothing effect on $\mathbf{W}$.

E Implementation Details

In this work, all quantization experiments are conducted on NVIDIA L20 GPUs, which feature 40GB of memory. The sequence length is set to 2048 for all evaluation tasks. In our rotation matrix experiments, we set the rotation block size to $B = 128$ and the number of sampling times to $K = 16$. Following the DuQuant method, we adopt a single zigzag permutation ($T = 1$) to improve efficiency. Before applying rotation, we perform a smoothing operation with

Table 8: The perplexity results (↓) of the LLaMA1, LLaMA2 and LLaMA3 model on the C4 dataset.

#Bit	Method	1-7B	1-13B	1-30B	2-7B	2-13B	3-8B
FP16	-	6.92	6.49	5.92	6.84	6.41	8.62
W4A4	DuQuant	7.60	7.00	6.26	7.63	6.89	11.45
	RUQuant	**7.57**	**6.91**	**6.25**	**7.62**	**6.87**	**11.39**
W6A6	DuQuant	6.96	6.52	5.49	6.89	6.44	8.78
	RUQuant	**6.95**	**6.51**	**5.94**	**6.88**	**6.43**	**8.75**

Algorithm 4 Rotation

`func Rotation(`$\mathbf{X}, \mathbf{W}, B, K, \lambda$`)`

shapeX ← **X**.shape
shapeW ← **W**.shape
X ← **X**.reshape(-1,B)
W ← **W**.reshape(B,-1)
Q ← $\mathbf{I}_{B\times B}$
for k = 1 to K **do**
  X ←Sample(**X**)
  H ← Aig 1(**x**)
  X ← **HX**
  Q ← **HQ**
  X ←Sample(**X**)
  R ← Aig 2(**x**, λ)
  X ← **RX**
  Q ← **GQ**
end for
X ← **X**.reshape(shapeX)
W ← **WQ**$^\top$.reshape(shapeW)
return **W**, **X**

a smoothing factor set to 0.6. During RTN quantization of the rotated weights and activations, we apply appropriate clipping: 1) For the W4A4 setting, We clip the maximum activation values in all projection blocks with a clipping ratio of 0.9 and clip the maximum values in the weight matrices with a clipping ratio of 0.8. 2) For the W6A6 setting, Since RUQuant yields better rotation performance in this case, we do not apply any clipping to weights or activations (clipping ratio set to 1). When fine-tuning the LH component, we only update the parameter $\boldsymbol{\theta}$, while keeping all other settings (e.g., clipping ratio and smoothing factor) the same as in the non-fine-tuned configuration. The fine-tuning is conducted with the following settings: epoch = 20, batch size = 1, and learning rate = 1×10^{-2}.

F Algorithm

Algorithm 3 shows the detailed process of RUQuant, and Algorithm 4 shows the Rotation function. Specifically, for each Transformer block $\mathcal{T}$, we perform the operations in two steps. In the first step, we apply the Householder and Givens Transformations to all linear layers within the Transformer block. Initially, the input $\mathbf{X}$ is divided into blocks of size B. Subsequently, all blocks share a common rotation matrix $\mathbf{Q}_c$, and through the composite linear transformation of Householder and Givens transforms, an exact mapping from non-uniform to uniform distribution is achieved. To reduce the variance between blocks, we further employ a zigzag permutation and a second rotation operation. In the second step, we initialize the Learnable Householder matrix parameters $\boldsymbol{\theta}$ using Algorithm 1 and fine-tune $\boldsymbol{\theta}$ based on the changes in block outputs before and after quantization. This process aims to further eliminate the impact of block division and align the quantization loss with the model loss.

Table 9: 5-shot MMLU performance of LLaMA2-7B and 13B under W4A4 quantization.

Model	Method	STEM	Hums	Social	Others
7B	FP16	36.88	43.25	51.74	52.41
	DuQuant	30.62	34.16	40.53	42.97
	RuQuant	**33.10**	**37.56**	**43.74**	**46.58**
13B	FP16	44.23	54.41	63.47	60.67
	DuQuant	39.10	46.48	**57.69**	55.64
	RuQuant	**39.96**	**47.37**	**57.69**	**55.89**

Table 10: 0-shot MMLU performance of LLaMA2-7B and 13B under W4A4 quantization.

Model	Method	STEM	Hums	Social	Others
7B	FP16	33.96	39.36	47.94	45.92
	DuQuant	29.92	30.24	36.30	35.38
	RuQuant	**30.88**	**32.86**	**39.65**	**36.18**
13B	FP16	42.35	47.76	60.71	59.47
	DuQuant	37.64	44.97	55.15	52.28
	RuQuant	**39.99**	**44.42**	**55.18**	**54.19**

G More Results

Table 8 shows the test results of RUQuant on the C4 dataset. As can be seen from the table, RUQuant still has certain advantages on the C4 dataset, and the performance of the quantized model also surpasses DuQuant. The experiments demonstrate that RUQuant consistently achieves lower perplexity across multiple model sizes and quantization levels.

Table 9 and 10 show the MMLU results under 0-shot and 5-shot settings. It can be seen that RUQuant achieves better performance than DuQuant across domains, further demonstrating its effectiveness on challenging downstream tasks.